

Prvo, tehnički deo je bio jasan i direktan. Ključni korak je bio prebacivanje sa 2D geometrije na 1D problem stacionaže, korišćenjem Shapely-jeve funkcije `project`. To je elegantno rešenje koje problem svodi na sortiranje i skeniranje. Definisao sam autoput liniju i projektovao svaku pumpu na nju, a zatim sortirao pumpe po stacionaži.

Ono što je najvažnije, implementirao sam dve Greedy strategije: "poslednji u dometu" i "prvi u dometu". Kao što smo i očekivali, "poslednji u dometu" je dao znatno bolje rezultate. Za  $D_{\max}$  od 50 km, dobio sam 7 punjača, dok je "prvi u dometu" zahtevao čak 21. Ovo lepo ilustruje zašto je Greedy algoritam, kada je dobro definisan, optimalan za ovaj tip problema.

Međutim, tu dolazimo do prvog interesantnog pitanja. Pokazujem na ekran ili skicu Algoritam je pronašao GAP na 296. kilometru, bez obzira na strategiju. To znači da, iako imamo pumpu na tom mestu, od nje do kraja autoputa nema nijedne druge pumpe u dometu od 50 km. Ovo nije problem samog algoritma, već je problem u podacima. Postavlja se pitanje: kako bismo mi, kao inženjeri, rešili ovaj problem u praksi? Da li bismo jednostavno prihvatili da poslednji deonica nije pokrivena, ili bismo morali da razmotrimo postavljanje novog punjača na lokaciji koja nije benzinska pumpa? Ovo je važna granica našeg rešenja – ono je potpuno zavisno od postojeće infrastrukture.

Dalje, kada pogledamo rezultate za  $D_{\max} = 30$  km, broj punjača raste na 12, što je očekivano. Ali, zanimljivo je da se i dalje pojavljuje isti GAP na kraju. To nam govori da problem nije u gustini pumpi na celoj deonici, već u distribuciji na samom kraju autoputa.

Ono što bih voleo da dodatno istražimo jeste robustnost ovog rešenja. Naš algoritam pretpostavlja da je svaka pumpa podjednako dobar kandidat, ali u stvarnosti, neke pumpe mogu imati veći kapacitet, bolji pristup ili jednostavno biti otvorene 24/7. Da li bismo mogli da implementiramo ponderisani Greedy algoritam, gde bi pored udaljenosti, birali i na osnovu nekog "kvaliteta" pumpe? To bi zadatak podiglo na viši nivo i približilo ga realnom svetu.

Takođe, vizuelizacija je korisna, ali bi mogla biti i interaktivna. Zamislite da možemo da "šetamo" duž autoputa i da nam se u realnom vremenu prikazuju koje su pumpe u dometu. To bi bio odličan alat za prezentaciju rešenja donosiocima odluka.

Na kraju, ono što je najvažniji zaključak iz ovog zadatka, a što bih želeo da naglasim, jeste da je naš "optimalni" rezultat od 7 punjača zapravo optimalan samo u kontekstu datih podataka. U realnosti, potrebno je uključiti i druge faktore: cenu izgradnje, protok saobraćaja, i, kao što smo videli, mogućnost proširenja mreže van postojećih pumpi. Zato Greedy algoritam vidim kao odličan polazni osnov, ali ne i kao konačno rešenje.

## 1. Rad sa GeoPandas i projekcijom

Prvi tehnički izazov bio je kako precizno projektovati tačke na liniju. Korišćenje `autoput_line.project(p)` iz Shapely-ja je bilo prirodno rešenje, ali sam morao da budem oprezan sa koordinatnim referentnim sistemom. Pošto je CRS bio EPSG:3857 (Web Mercator), dužine su u metrima, što je pogodno jer radimo sa `D_max` od 50 km. Da je CRS bio u stepenima, morao bih da vršim transformaciju.

Ovo je dobro mesto da zastaneš i pitaš profesora: "Da li biste preporučili neku drugu projekciju za ovakav zadatak, s obzirom da EPSG:3857 ima poznata izobličenja na većim razdaljinama?"

## 2. Sortiranje i indeksiranje

Nakon projektovanja, ključno je bilo sortirati pumpe po stacionaži. Iskoristio sam `pumpe.sort_values("stacionaza").reset_index(drop=True)` da bih dobio čist indeks. Ovo je važno jer Greedy algoritam pretpostavlja da su kandidati uređeni duž putanje.

Mogao sam da koristim i `pumpe = pompe.sort_values("stacionaza")` bez resetovanja indeksa, ali bi onda pristup preko `iloc` bio manje čitljiv. Ovo je mala, ali bitna odluka za čitljivost koda.

## 3. Dve strategije u jednoj funkciji?

U početku sam razmišljao da napravim jednu funkciju sa parametrom `mode="last"` ili `mode="first"`, ali sam na kraju napravio dve odvojene funkcije: `greedy_punjaci_last` i `greedy_punjaci_first`. Zašto? Zato što je razlika samo u jednoj liniji koda (`validni.iloc[-1]` vs `validni.iloc[0]`), ali odvojene funkcije su:

- Lakše za testiranje
- Čitljivije u kodu
- Omogućavaju nezavisnu modifikaciju ako neka strategija zahteva dodatnu logiku

Da li je ovo preterano? Možda, ali smatram da je dobra praksa.

## 4. Upravljanje GAP-om

Jedna od ključnih stvari koju sam morao da rešim je kako algoritam reaguje kada nema kandidata u dometu. U mom kodu, `if validni.empty:` prekida petlju i ispisuje upozorenje. Ovo je važno jer:

- Sprečava beskonačnu petlju (`while prethodno_km < autoput_length`)
- Jasno ukazuje na problem u podacima
- Omogućava da vidimo gde tačno postoji problem

Alternativa bi bila da algoritam "preskoči" taj deo i nastavi dalje, ali to bi dalo lažnu sliku o pokrivenosti. Ovo sam svesno odabrao kao konzervativniji pristup.

## 5. Vizuelizacija i debug

Kod vizuelizacije, najveći izazov bio je kako prikazati GAP-ove na mapi. Odabrao sam da ih označim crvenim anotacijama na sredini između dve tačke:

python

```
mid_x = (p1.geometry.x + p2.geometry.x) / 2
```

```
mid_y = (p1.geometry.y + p2.geometry.y) / 2
```

```
ax.annotate(f"GAP {gap:.1f} km", (mid_x, mid_y), ...)
```

Ovo je jednostavno, ali efikasno. Takođe sam koristio `izabrani_nazivi` set za brzo proveravanje da li je pumpa izabrana, umesto da ponovo prolazim kroz listu izabranih.

Još jedno pitanje za profesora: "Da li biste preporučili neki drugi način za vizuelizaciju GAP-ova, možda korišćenjem `plt.axvspan` ili slično, ako bismo radili u 1D prikazu?"

#### 6. Upotreba `uv` za upravljanje paketima

Koristio sam `uv` umesto klasičnog `pip` + `venv` jer je brži i jednostavniji. `uv sync` automatski instalira sve zavisnosti iz `pyproject.toml`. Ovo je dobra praksa za reproduktivnost - profesor može da pokrene moj kod bez razmišljanja o verzijama paketa.

Da sam koristio `pip freeze > requirements.txt`, to bi bilo manje elegantno. `uv` je moderna praksa koju sam želeo da demonstriram.

#### 7. Performanse

Sa 22 pumpe, performanse nisu problem - algoritam se izvršava trenutno. Ali da imamo 22.000 pumpi, `apply` sa `project` bi mogao da bude spor. U tom slučaju bih koristio vektorizaciju ili `shapely.ops.transform` za batch obradu. Ovde nije bilo potrebe, ali je dobro razmišljati o skalabilnosti.

#### 8. Testiranje

Iako nisam pisao formalne testove, ručno sam proverio nekoliko scenarija:

- Da li algoritam bira poslednju pumpu u dometu? (Provera: ručno računanje za prvih nekoliko koraka)
- Da li se GAP ispravno detektuje? (Provera: na kraju autoputa)
- Da li obe strategije daju očekivane rezultate? (Provera: `len()` upoređenje)

Ovo je dobro mesto za pitanje: "Da li biste u ovakvom zadatku očekivali da napišemo unit testove, ili je ručna provera dovoljna?"

O upotrebi `iloc` vs `loc`:

Koristim `validni.iloc[-1]` i `validni.iloc[0]` jer radimo sa `DataFrame`-om koji ima numerički indeks (posle `reset_index`). Da sam koristio `loc`, morao bih da znam tačan indeks, što je manje fleksibilno.

O anotacijama na mapi:

Koristio sam `xytext=(5, -12)` da pomerim labelu van tačke, jer se tačke preklapaju na karti. Ovo je mali UX detalj koji poboljšava čitljivost.

O `tight_layout()`:

Dodao sam `plt.tight_layout()` da izbegnem sečenje labela na ivicama figure. Ovo je često zanemareno, ali bitno za kvalitetnu vizuelizaciju.